

목차

1장 서버와 요청-응답 모델	2
1.1 서버의 정의와 수신 대기	2
1.2 클라이언트와 서버의 역할	3
1.3 요청-응답의 대응 관계	4
1.4 외부 계약과 내부 구현의 분리	5
1.5 요청-응답 처리 개요	6
1.6 요청 처리 구간과 소요 시간	7
1.7 서버 경계 핵심 정리	8
2장 요청 메시지의 구성	9
2.1 구조화된 요청 메시지	9
2.2 요청 동작과 처리 의도	10
2.3 요청 경로와 처리 진입점	11
2.4 헤더의 역할과 부가 정보	12
2.5 본문의 역할과 업무 데이터	13
2.6 전송 메타데이터와 업무 데이터 분리	14
2.7 요청 해석 절차	15
3장 라우팅과 처리자 선택	16
3.1 라우터의 책임	16
3.2 경로표와 라우팅 규칙	17
3.3 동작과 경로의 결합 비교	18
3.4 동적 경로 구간과 값 추출	19
3.5 라우팅 결정 절차	20
3.6 라우팅 실패 응답	21
3.7 중복 라우팅 규칙과 우선순위	22
4장 요청 처리와 응답 생성	23
4.1 요청 처리자의 책임	23
4.2 요청 맥락의 구성과 전달	24
4.3 내부 처리 결과와 응답의 구분	25
4.4 응답 메시지의 구성	26
4.5 상태와 본문의 의미 일치	27
4.6 오류 응답의 정보 노출 범위	28
4.7 요청 처리와 응답 생성 절차	29
5장 동시 요청 처리와 자원 관리	30
5.1 동시 요청의 정의	30
5.2 작업자와 요청 작업의 관계	31
5.3 처리 대기열과 대기 시간	32
5.4 요청 맥락 격리와 공유 상태	33
5.5 동시 요청 처리 시간선	34
5.6 대기 작업과 계산 작업의 차이	35
5.7 과부하와 수용 한계	36
6장 무상태 서버와 상태 관리	37
6.1 무상태 서버의 정의	37
6.2 서버 인스턴스 종속 상태의 문제	38
6.3 업무 상태의 외부 저장	39
6.4 요청 식별자와 업무 상태의 구분	40
6.5 무상태 서버의 수평 확장	41
6.6 재시도와 중복 요청 처리	42
6.7 무상태 서버 구성	43
7장 전체 요청 처리 흐름	44
7.1 전체 요청 처리 단계	44
7.2 조회 요청의 정상 처리 사례	45
7.3 존재하지 않는 경로의 처리 사례	46
7.4 동시 요청 처리 사례	47
7.5 서버 인스턴스 변경 사례	48
7.6 요청 처리 흐름 점검 항목	49
7.7 핵심 원칙 정리	50

1.1 서버의 정의와 수신 대기

서버는 특정한 크기나 형태의 컴퓨터가 아니라 네트워크 통신에서 수행하는 역할을 뜻한다. 프로그램이 지정된 통신 지점에서 요청을 수신하고, 요청의 의미를 해석한 뒤, 처리 결과를 응답으로 반환하면 서버 역할을 수행한다. 이 역할은 개인용 컴퓨터의 단일 프로세스에서도 실행할 수 있고 여러 컴퓨터에 분산할 수도 있다. 배치 위치와 규모가 달라도 외부에 제공하는 요청-응답 계약은 동일한 기준으로 설명할 수 있다.

실행 중인 모든 프로그램이 서버는 아니다. 서버 역할의 핵심은 지정된 통신 지점에서 요청을 수신 대기하는 데 있다. 서버는 지원하는 메시지 구조, 요청 동작, 경로, 응답 형식을 외부 계약으로 정의한다. 계약에 맞는 요청이 도착하면 처리를 시작하고, 지원하지 않는 요청에는 그 사실을 나타내는 응답을 반환한다. 수신 대기과 계약 기반 처리가 일반 실행 프로그램과 구분되는 핵심 특성이다.

서버 구조를 분석할 때는 내부 함수 수보다 요청 주체, 수신 지점, 메시지 형식, 응답 의미를 먼저 확인해야 한다. 이 항목은 서버 외부와 내부를 나누는 시스템 경계를 구성한다. 내부 저장 방식이나 함수 구성이 변경되더라도 외부 계약이 유지되면 클라이언트는 동일한 방식으로 서버를 사용할 수 있다. 이후 절에서는 요청이 이 경계를 통과해 처리되고 응답으로 반환되는 순서를 단계별로 설명한다.

1.2 클라이언트와 서버의 역할

서버에 요청을 전송하는 쪽을 클라이언트라고 부른다. 화면의 버튼을 누른 브라우저일 수도 있고, 정해진 시각에 자료를 가져오는 다른 프로그램일 수도 있다. 중요한 것은 장치의 종류가 아니라 요청을 시작한 역할이다. 클라이언트는 자신이 원하는 일을 요청에 담아 보내고, 서버는 그 요청을 읽은 뒤 결과를 응답으로 반환한다. 이 한 쌍이 가장 작은 통신 단위다.

클라이언트와 서버라는 이름은 영구적인 신분이 아니다. 한 프로그램이 사용자에게는 서버로 답하면서, 필요한 자료를 얻기 위해 다른 서버에 요청을 보낼 수도 있다. 그 순간 같은 프로그램은 한쪽 관계에서는 서버이고 다른 관계에서는 클라이언트다. 역할을 관계마다 나누어 보면 여러 프로그램이 이어진 구조에서도 누가 먼저 무엇을 요구했고, 어느 답이 어느 요청에 대한 것인지 추적하기 쉬워진다.

요청은 일방적인 명령과도 다르다. 서버는 요청을 받았다고 해서 반드시 원하는 결과를 내놓을 수 있는 것은 아니다. 경로를 찾지 못하거나, 지금 허용되지 않은 동작이거나, 내부 작업이 실패했다는 사실도 응답으로 알려야 한다. 성공과 실패를 모두 답으로 다루면 통신은 침묵 대신 명확한 결과를 남긴다. 이 책에서는 요청 하나가 어떤 판단을 거쳐 그에 대응하는 응답 하나로 닫히는지 계속 살펴본다.

1.3 요청-응답의 대응 관계

요청을 보낸 뒤에는 그 요청에 대한 응답이 돌아와야 한 차례의 통신이 끝난다. 서버가 자료를 잘 만들었더라도 엉뚱한 요청에 그 결과를 붙여 보내면 올바른 통신이 아니다. 그래서 통신을 관찰할 때는 받은 요청 수와 보낸 응답 수만 세지 않고, 각각의 응답이 어떤 요청에서 시작되었는지 연결해서 본다. 요청과 응답의 대응 관계가 서버 흐름을 이해하는 기준선이 된다.

한 번에 요청 하나만 오면 이 연결은 당연해 보인다. 하지만 여러 사용자가 거의 같은 시각에 서로 다른 자료를 요구하면 서버 안에는 여러 작업이 동시에 존재한다. 먼저 들어온 요청이 늦게 끝날 수도 있고, 나중 요청이 빠르게 응답할 수도 있다. 완료 순서가 도착 순서와 달라도 각 결과가 원래 요청으로 돌아가야 한다. 이 원칙 덕분에 동시 처리에서도 요청-응답 관계가 뒤섞이지 않는다.

응답이 오지 않는 상황도 대응 관계로 해석해야 한다. 서버가 작업 중인지, 통로가 끊겼는지, 응답이 중간에서 사라졌는지 클라이언트는 처음부터 알 수 없다. 그래서 기다릴 최대 시간을 정하고, 시간이 지나면 그 요청은 완료되지 않았다고 판단한다. 이후 다시 요청할지는 별도의 결정이다. 핵심은 한 요청의 처리 주기가 응답을 받거나 기다림을 끝내는 판단으로 닫혀야 후속 처리를 안전하게 고를 수 있다는 점이다.

1.4 외부 계약과 내부 구현의 분리

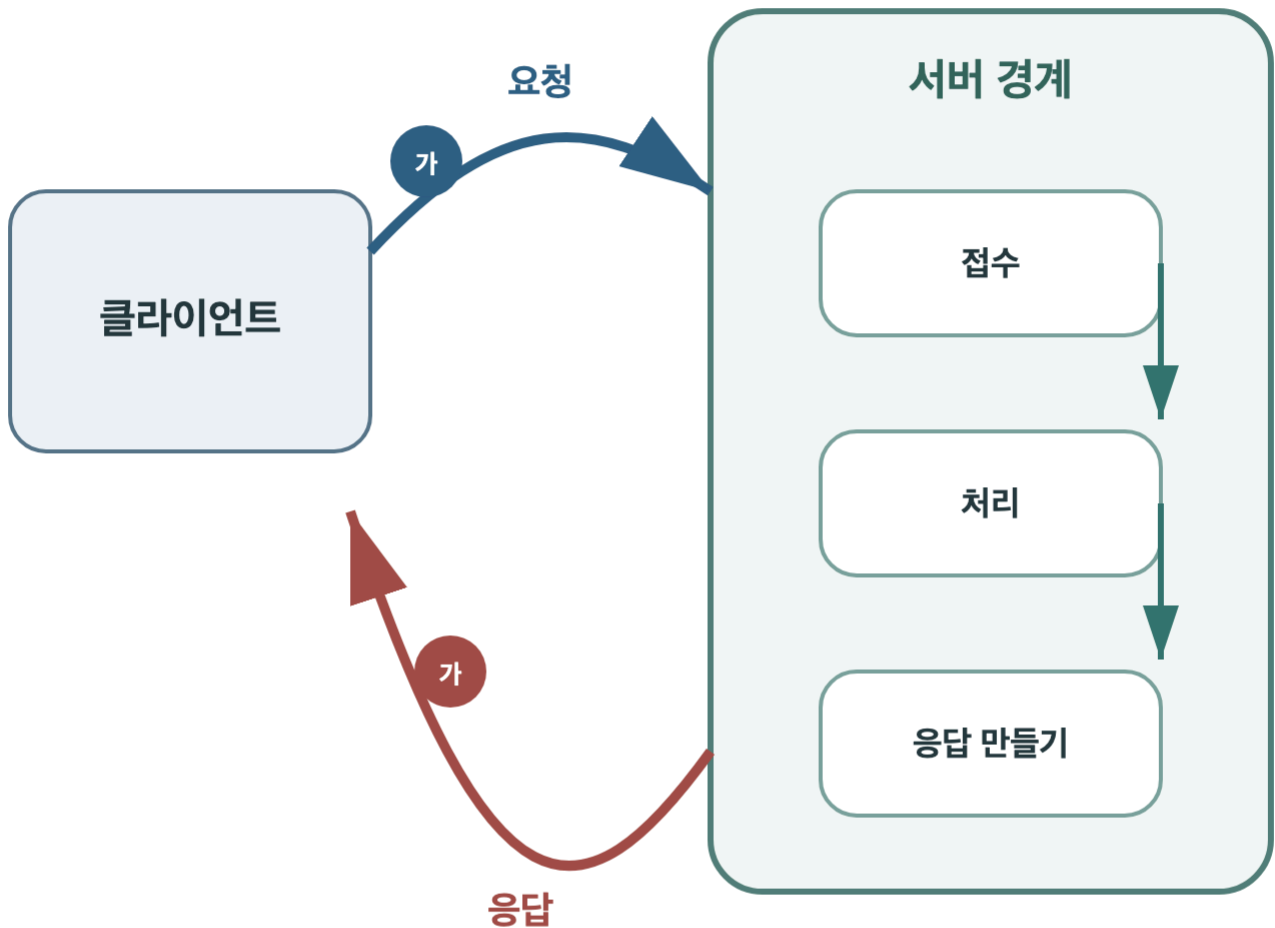
서버를 처음 분석할 때 내부 함수와 파일 구조부터 모두 확인하려는 접근은 효율적이지 않다. 내부를 살피는 일은 구현자에게 중요하지만, 서버 밖의 클라이언트가 그 세부에 의존해서는 안 된다. 클라이언트가 알아야 할 것은 어떤 요청을 보내면 어떤 의미의 응답을 받을 수 있는지다. 내부의 순서와 저장 방식은 그 약속을 지키는 한 바뀔 수 있다.

이 구분이 없으면 작은 수정도 연쇄적인 문제가 된다. 서버가 자료를 찾는 방법을 바꾸었을 뿐인데 클라이언트가 내부 이름을 직접 알고 있었다면 함께 수정해야 한다. 반대로 외부 계약이 안정되어 있으면 서버는 같은 응답 의미를 유지하면서 내부 작업을 더 단순하게 고칠 수 있다. 경계는 정보를 숨기기 위한 벽이라기보다, 서로 다른 프로그램이 꼭 알아야 할 약속만 공유하게 하는 선이다.

물론 내부 오류를 무조건 감추라는 뜻은 아니다. 실패 종류를 클라이언트가 후속 처리를 고를 수 있는 수준으로 알려야 한다. 다만 내부 파일 경로나 구체적인 처리 단계까지 그대로 노출하는 것은 계약을 불필요하게 넓힌다. 좋은 응답은 바깥에서 필요한 의미를 충분히 주되 내부 구조에 대한 의존은 만들지 않는다. 이 원칙을 적용하면 이후 라우팅과 처리 단계도 외부 경계와 내부 작업으로 나누어 볼 수 있다.

요청-응답 처리 개요

클라이언트 요청이 서버 처리 후 응답으로 반환되는 과정



요청 식별자는 응답 반환까지 유지된다

외부에는 계약을 노출하고 내부 구현은 서버 경계 안에 둔다

1.6 요청 처리 구간과 소요 시간

버튼을 누른 뒤 화면이 곧바로 바뀌면 모든 일이 한순간에 일어난 것처럼 보인다. 그러나 짧은 요청-응답 처리도 여러 구간으로 나눌 수 있다. 클라이언트가 요청을 만들고 통로로 보내는 시간, 메시지가 서버 경계에 도착하는 시간, 서버가 내용을 처리하는 시간, 만든 응답이 다시 클라이언트로 돌아오는 시간이 차례로 존재한다. 화면은 이 구간을 합친 결과만 보여 준다.

구간을 나누는 까닭은 책임과 원인이 다르기 때문이다. 요청이 서버에 도착하기 전 통로에서 오래 머물렀다면 내부 처리 코드를 빨리 고쳐도 체감 시간은 달라지지 않는다. 반대로 서버 안의 특정 작업이 오래 걸린다면 전송 통로만 넓혀도 해결되지 않는다. 요청-응답 구간을 하나로 합쳐 보면 느리다는 현상만 남지만, 시간선을 펼치면 어디까지 도착했고 어느 단계에서 기다렸는지 질문할 수 있다.

이 책은 실제 시간을 측정하는 도구를 깊게 다루지는 않는다. 대신 흐름을 그릴 때 항상 경계를 표시한다. 요청 생성, 서버 접수, 경로 선택, 작업 실행, 응답 생성, 반환이라는 칸을 두면 작은 서버도 관찰 가능한 사건들의 연속이 된다. 이후 새로운 개념을 배울 때도 이 시간선의 어느 칸에 속하는지 놓아 보면, 서로 다른 문제를 섞지 않고 이해할 수 있다.

1.7 서버 경계 핵심 정리

첫 장에서 서버는 거대한 장비가 아니라 요청을 기다리고 응답하는 역할로 정의되었다. 클라이언트는 요청을 시작하고, 서버는 그 요청에 대응하는 응답으로 한 차례의 통신을 닫는다. 같은 프로그램도 관계에 따라 두 역할을 번갈아 맡을 수 있다. 따라서 이름보다 누가 먼저 메시지를 보냈고 누가 그 메시지에 답하는지를 보는 편이 흐름을 정확하게 설명한다.

또한 서버의 바깥과 안쪽을 구분했다. 외부 계약에는 요청을 보내는 방법과 응답의 의미가 드러나지만, 내부 함수와 저장 방식까지 포함할 필요는 없다. 이 경계가 안정적이면 내부 구현을 바꾸면서도 클라이언트와의 약속을 지킬 수 있다. 실패 응답 역시 내부 사정을 모두 노출하는 보고서가 아니라, 바깥에서 후속 처리를 고를 수 있게 하는 계약의 일부다.

이제 다음 장에서는 경계를 통과하는 요청 메시지를 펼쳐 본다. 요청이 하나의 글덩이가 아니라 의도와 목적지와 부가 정보와 선택적 내용으로 구성된다는 사실을 알면, 서버가 어떤 순서로 메시지를 읽는지 이해할 수 있다. 아직 자료의 옳고 그름이나 복잡한 내부 구조로 들어가지는 않는다. 먼저 서버가 요청을 식별하는 데 필요한 구성 요소부터 차례로 설명한다.

2.1 구조화된 요청 메시지

요청은 서버가 해석할 수 있도록 미리 정의된 구조를 가진 메시지다. 자유 형식 문자열 하나로 모든 의도를 전달하면 서버가 동작, 대상, 부가 조건, 처리 데이터를 안정적으로 구분하기 어렵다. 따라서 요청은 동작, 경로, 헤더, 선택적 본문처럼 책임이 다른 필드로 구성한다. 서버와 클라이언트는 각 필드의 위치와 의미를 동일한 통신 규약으로 공유해야 한다.

서버는 동작과 경로를 사용해 요청의 처리 진입점을 결정한다. 헤더는 본문 표현 형식이나 요청 맥락 같은 전송 메타데이터를 제공하고, 본문은 생성이나 변경에 필요한 업무 데이터를 전달한다. 조회 요청은 본문이 없을 수 있으며, 생성이나 변경 요청은 여러 값을 본문에 포함할 수 있다. 각 필드의 필수 여부와 누락 시 응답은 외부 계약에 명시해야 한다.

요청 구조를 이해할 때는 필드 이름만 암기하지 않고 각 필드가 사용되는 처리 단계를 연결해야 한다. 동작과 경로는 라우팅 입력이고, 헤더는 메시지 해석과 통신 맥락에 사용되며, 본문은 처리자가 수행할 업무의 입력이 된다. 책임을 구분하면 요청 해석 실패와 업무 처리 실패를 서로 다른 단계로 식별할 수 있다. 다음 절부터 각 구성 요소의 역할과 주의점을 구체적으로 설명한다.

2.2 요청 동작과 처리 의도

같은 자료를 가리키더라도 클라이언트가 원하는 행동은 다를 수 있다. 목록을 보고 싶은 경우와 새 항목을 추가하고 싶은 경우, 기존 값을 바꾸고 싶은 경우는 목적지가 같아도 결과가 다르다. 요청의 동작 칸은 이 의도를 짧게 압축한다. 서버는 목적지만 보지 않고 동작을 함께 읽어 조회로 보낼지 변경 작업으로 보낼지 구분한다.

동작은 결과를 보장하는 말이 아니다. 변경을 원한다는 동작을 보냈다고 실제 변경이 반드시 이루어지는 것은 아니다. 서버가 해당 경로에서 그 동작을 받지 않거나, 요청자가 필요한 조건을 갖추지 못했거나, 작업 중 문제가 생길 수 있다. 동작은 클라이언트가 무엇을 시도하는지 나타내고, 성공 여부와 구체적인 결과는 응답이 알려 준다. 의도와 결과를 나누면 실패를 더 정확히 설명할 수 있다.

또한 동작을 아무렇게나 섞으면 서버와 클라이언트가 같은 요청을 다르게 이해한다. 조회 의도를 가진 요청이 실제로 자료를 바꾸면 다시 시도하거나 미리 불러오는 동작이 위험해진다. 따라서 동작 이름의 의미를 일관되게 사용해야 한다. 이 절에서는 특정 표기법을 외우기보다, 동작이 요청자의 의도를 서버의 첫 판단에 전달하는 계약이라는 사실을 확인하면 충분하다.

2.3 요청 경로와 처리 진입점

요청의 목적지는 보통 경로로 표현된다. 경로는 서버 안의 실제 폴더 주소가 아니라, 외부에서 어떤 종류의 대상을 다루려는지 나타내는 공개된 처리 진입점이다. 예를 들어 여러 책을 가리키는 처리 진입점과 특정 책 하나를 가리키는 처리 진입점을 다르게 둘 수 있다. 클라이언트는 내부 저장 위치를 몰라도 약속된 경로를 통해 원하는 처리로 들어간다.

경로에는 고정된 부분과 요청마다 달라지는 부분이 함께 있을 수 있다. 대상 종류를 나타내는 말은 고정하고, 특정 대상을 구분하는 값만 바꾸는 식이다. 서버는 전체 문자열을 무작정 비교하지 않고 경로의 모양을 읽어 어느 규칙에 맞는지 판단한다. 달라지는 값을 찾으면 이후 처리에서 쓸 수 있도록 요청 맥락에 담는다. 이 과정이 다음 장의 라우팅과 직접 이어진다.

좋은 경로는 내부 구현을 그대로 복사하지 않는다. 저장소의 표 이름이나 함수 이름이 경로가 되면 내부 변경이 외부 계약까지 흔든다. 대신 클라이언트가 다루는 대상과 관계를 안정된 말로 표현한다. 경로는 목적지를 알려 주되, 그 뒤에서 어떤 코드가 실행되는지는 감춘다. 요청 경계가 하는 일은 외부의 의미를 내부 처리의 처리 진입점으로 연결하는 데 있다.

2.4 헤더의 역할과 부가 정보

요청에는 동작과 경로만으로 설명하기 어려운 맥락이 있다. 본문이 어떤 형식으로 적혔는지, 클라이언트가 어떤 형태의 응답을 이해할 수 있는지, 한 차례의 통신을 구분하기 위한 표식이 무엇인지 같은 정보다. 이런 부가 정보는 본문과 분리된 헤더 칸에 담긴다. 서버는 본문을 깊이 읽기 전에 헤더를 보고 메시지를 어떻게 해석할지 준비할 수 있다.

헤더는 모든 것을 넣는 범용 데이터 영역이 아니다. 실제로 처리할 자료 전체를 헤더에 넣으면 크기와 표현 제약이 뒤섞이고, 어떤 정보가 메시지 설명인지 업무 내용인지 구분하기 어렵다. 헤더에는 메시지를 다루는 방법과 통신 맥락을, 본문에는 작업에 필요한 내용을 두는 편이 경계를 선명하게 만든다. 두 칸의 책임이 다르다는 사실이 중요하다.

서버는 필요한 헤더가 없거나 이해할 수 없는 값이 들어오면 그 요청을 처리할 수 없다고 답할 수 있다. 반대로 선택적인 헤더라면 합리적인 기본 동작을 택할 수 있다. 어느 정보가 필수인지는 서버의 외부 계약에 속한다. 클라이언트가 보낸 부가 정보를 무시하거나 임의로 추측하지 않고, 약속된 의미에 따라 읽는 것이 요청 경계를 안정적으로 유지하는 방법이다.

2.5 본문의 역할과 업무 데이터

새 자료를 만들거나 값을 바꾸려면 목적지와 동작만으로는 부족하다. 무엇을 저장하고 어떤 값으로 바꿀지 실제 업무 데이터가 필요하다. 요청 본문은 이 내용을 담는 영역이다. 여러 값을 이름과 값의 짝으로 표현할 수도 있고, 순서가 있는 항목으로 보낼 수도 있다. 중요한 것은 클라이언트와 서버가 본문의 표현 방식을 함께 약속한다는 점이다.

모든 요청에 본문이 필요한 것은 아니다. 이미 존재하는 자료를 조회할 때는 경로만으로 대상을 충분히 가리킬 수 있다. 반대로 본문이 필요한 동작에서 내용이 비어 있으면 서버는 작업을 시작할 재료가 없다. 본문 유무를 동작과 함께 보면 메시지의 의도가 더 분명해진다. 단순히 칸이 존재하는지보다 그 동작에서 어떤 의미를 갖는지 살펴야 한다.

헤더와 본문을 연결해서 보면 요청 메시지의 구조가 명확해진다. 헤더는 본문을 어떤 방식으로 해석해야 하는지 알려 주고, 본문은 실제 읽을 내용을 제공한다. 표현 형식을 알리는 설명과 표현된 자료 자체를 분리하면 서버가 어디에서 해석에 실패했고 어디에서 업무 조건이 맞지 않았는지 구별하기 쉬워진다. 자세한 데이터 검증은 다른 주제지만, 메시지 경계가 업무 데이터를 올바르게 넘기는 출발점이라는 점은 여기서 확인할 수 있다.

2.6 전송 메타데이터와 업무 데이터 분리

작은 서버를 빠르게 만들다 보면 모든 값을 본문 하나에 넣고 싶어진다. 통신을 구분하는 표식, 본문 형식, 실제 저장할 값이 한곳에 모이면 처음에는 간단해 보인다. 그러나 서버는 어떤 값이 메시지를 해석하기 위한 것인지, 어떤 값이 업무 작업에 쓰이는지 매번 구별해야 한다. 통신 규칙이 바뀔 때 업무 자료까지 함께 수정되는 문제도 생긴다.

반대로 모든 값을 헤더에 넣는 것도 해답이 아니다. 헤더는 메시지에 관한 설명을 빠르게 읽도록 만든 영역이다. 크고 복잡한 업무 데이터를 넣으면 네트워크 중간 장치가 기대하는 방식과 어긋날 수 있고, 내용을 다루는 도구도 불편해진다. 어느 한 칸을 만능으로 만들기보다 정보가 왜 존재하는지에 따라 자리를 나누는 편이 요청을 읽는 순서를 명확하게 한다.

경계를 나누면 실패도 구체적으로 말할 수 있다. 본문의 표현 방식을 알 수 없는 문제와, 표현은 읽었지만 필요한 업무 값이 없는 문제는 서로 다르다. 클라이언트는 어느 부분을 고쳐야 할지 알 수 있고 서버는 처리 단계에 들어가기 전 메시지 문제를 닫을 수 있다. 전송 메타데이터와 업무 데이터의 구분은 형식 취향이 아니라 책임과 오류 위치를 분리하는 장치다.

2.7 요청 해석 절차

서버 경계에 요청이 도착하면 먼저 약속된 메시지 구조로 읽을 수 있는지 확인한다. 그다음 동작과 경로를 살펴 어느 처리 진입점으로 보내야 하는지 판단한다. 필요한 부가 정보는 헤더에서 얻고, 실제 업무 데이터가 있다면 본문을 해석한다. 구현마다 세부 순서는 다를 수 있지만, 메시지 설명과 처리 재료를 구분한다는 원리는 같다.

이 순서에서 중요한 것은 아직 실제 업무를 수행하지 않았다는 점이다. 서버는 요청이 무엇을 뜻하는지 파악하고 알맞은 처리자를 찾을 준비를 했을 뿐이다. 이 경계에서 문제가 발견되면 업무 처리 단계로 진행하지 않고 이해할 수 없는 요청이라는 응답을 만들 수 있다. 불필요한 작업을 줄이고 오류의 위치를 선명하게 하는 효과가 있다.

다음 장에서는 동작과 경로가 어떻게 처리자를 선택하는지 살펴본다. 이를 라우팅이라고 부른다. 요청을 읽는 단계와 실제 업무 처리를 곧바로 이어 붙이면 어느 규칙이 처리 진입점을 골랐는지 보이지 않는다. 라우팅을 독립된 판단으로 떼어 놓으면 경로가 없을 때와 처리 중 실패했을 때를 구분할 수 있고, 같은 서버 안의 여러 처리 진입점을 질서 있게 관리할 수 있다.

3.1 라우터의 책임

한 서버는 목록 조회, 단건 조회, 생성, 변경처럼 여러 종류의 요청을 처리할 수 있다. 요청마다 실행할 처리자가 다르므로 동작과 경로를 기준으로 처리자를 선택하는 단계가 필요하다. 라우터는 수신한 요청의 동작과 경로를 등록된 라우팅 규칙과 비교하고, 조건이 일치하는 하나의 처리자를 선택한다. 이 선택 과정이 라우팅의 핵심 기능이다.

라우터는 자료 조회, 계산, 상태 변경 같은 업무 결과를 직접 생성하지 않는다. 라우터의 책임은 요청을 분류하고 처리자에 전달하는 데 한정된다. 실제 업무는 선택된 처리자가 수행한다. 책임을 분리하면 경로가 존재하지 않는 문제, 요청 동작이 허용되지 않는 문제, 처리자 실행 중 발생한 문제를 서로 다른 단계의 오류로 구분할 수 있다.

소규모 서버에서는 경로 비교와 업무 처리를 하나의 함수에 작성해도 동작할 수 있다. 그러나 공개 경로가 늘어나면 조건문의 우선순위와 중복 여부를 파악하기 어려워진다. 라우팅 규칙을 별도로 관리하면 서버가 제공하는 처리 진입점을 일관된 형식으로 확인할 수 있다. 처리자는 전체 경로표를 알 필요 없이 자신에게 전달된 요청의 업무 규칙에 집중할 수 있다.

3.2 경로표와 라우팅 규칙

라우터가 비교할 기준은 경로표에 모인다. 경로표의 한 줄에는 받을 동작, 경로의 모양, 연결할 처리자가 함께 적힌다. 요청이 오면 라우터는 동작과 경로가 맞는 줄을 찾고 그 줄에 연결된 처리를 선택한다. 표에 없는 조합은 서버가 공개한 처리 진입점이 아니므로 임의의 비슷한 처리로 보내지 않는다.

경로표는 내부 실행 설정이면서 외부 약속의 목록이기도 하다. 클라이언트에게 안내한 처리 진입점은 이 표와 맞아야 하고, 표에서 경로를 없애면 기존 클라이언트가 더는 같은 요청을 보낼 수 없다. 그래서 경로를 추가하거나 바꾸는 일은 단순한 함수 이름 변경보다 영향이 크다. 서버 경계의 계약을 수정하는 일이기 때문이다.

표를 읽을 때는 경로 문자열만 보지 않는다. 같은 경로라도 동작이 다르면 다른 줄이 될 수 있고, 고정 경로와 값이 들어가는 경로가 서로 겹칠 수도 있다. 등록된 순서에만 기대면 작은 수정으로 결과가 바뀔 수 있다. 규칙이 서로 구분되는지 확인하고, 겹친다면 어떤 규칙이 더 구체적인지 명확히 정해야 한다.

3.3 동작과 경로의 결합 비교

라우팅은 경로만 비슷하다고 끝나지 않는다. 요청의 동작과 경로가 한 경로표 줄에 함께 맞아야 한다. 같은 책 처리 진입점을 조회하는 요청과 내용을 바꾸는 요청은 경로가 같아도 다른 처리자에게 갈 수 있다. 라우터가 동작을 무시하면 조회하려던 요청이 변경 처리로 들어가거나, 변경 의도가 조회 결과로 끝나는 위험이 생긴다.

비교 결과는 세 가지로 나눠 생각할 수 있다. 동작과 경로가 모두 맞으면 해당 처리자로 보낸다. 경로 모양은 있지만 그 동작을 받는 줄이 없다면 목적지는 알지만 행동을 지원하지 않는 경우다. 경로 모양 자체가 없다면 서버가 그 목적지를 모르는 경우다. 두 실패는 클라이언트가 고칠 부분이 다르므로 같은 답으로 뭉개지 않는 편이 좋다.

라우팅을 이런 비교 과정으로 보면 자동으로 보이는 연결이 단순해진다. 서버가 요청 이름을 보고 함수를 알아서 찾는 것이 아니라, 미리 등록한 표와 동작·경로를 대조해 한 줄을 선택하는 것이다. 규칙이 명시되어 있으면 새로운 처리 진입점을 추가할 때도 기존 조합과 겹치는지 확인할 수 있고, 예상과 다른 처리자가 선택된 원인도 경로표에서 추적할 수 있다.

3.4 동적 경로 구간과 값 추출

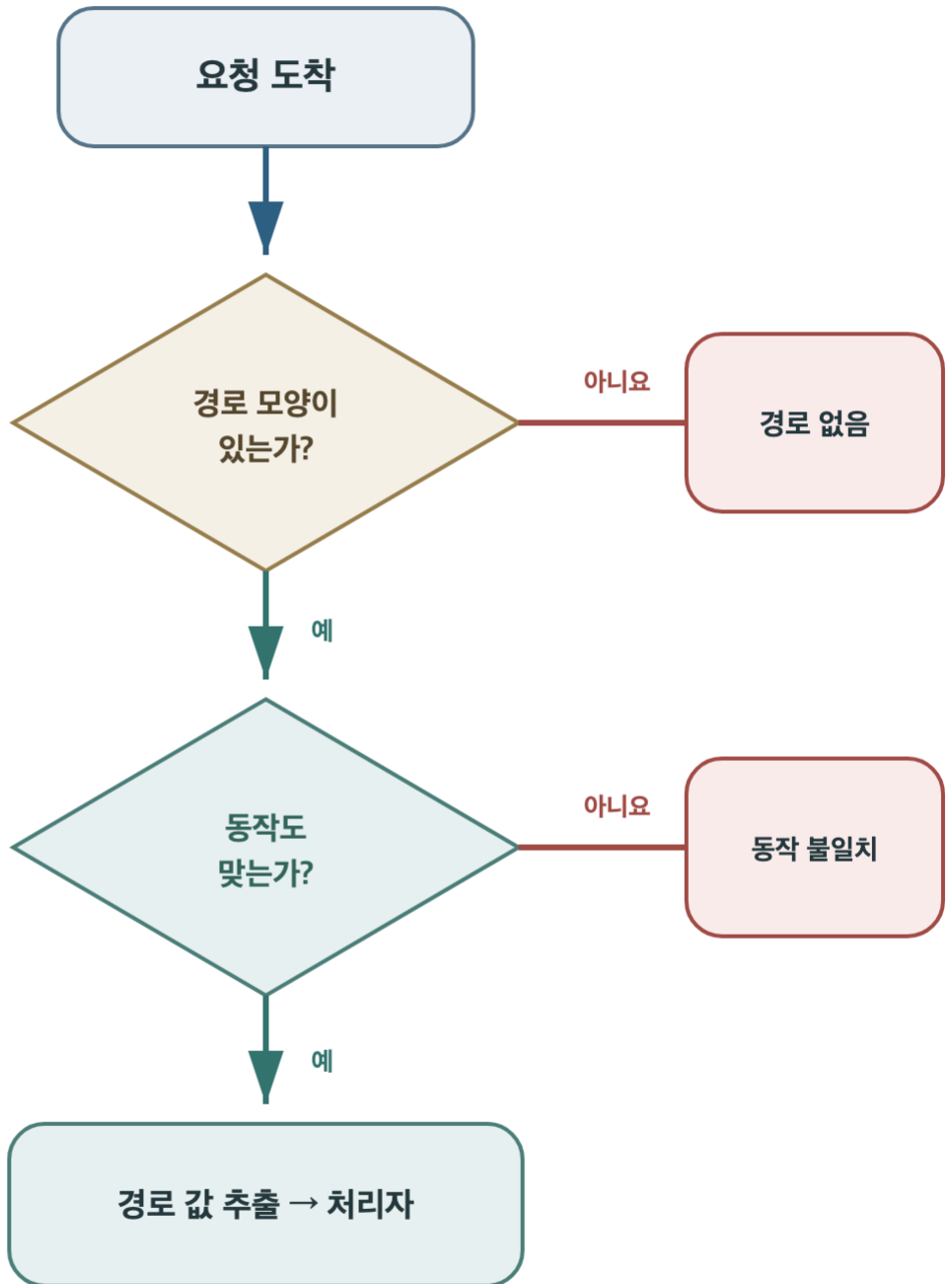
특정 책마다 경로표 줄을 하나씩 만들 수는 없다. 대상 수가 늘 때마다 표도 끝없이 늘어나기 때문이다. 대신 경로의 일정한 자리에 변하는 값이 들어올 수 있다고 표시한다. 책을 가리키는 고정 구간 뒤에 식별자가 오는 모양을 등록하면, 여러 책 요청을 한 규칙으로 받을 수 있다. 이 변하는 자리를 경로 변수라고 생각하면 된다.

라우터는 요청 경로가 패턴에 맞는지 비교하면서 실제 들어온 값을 꺼낸다. 꺼낸 값은 이후 처리자가 사용할 수 있도록 요청 맥락에 담긴다. 여기서 라우터의 책임은 위치를 식별하고 값을 전달하는 데 있다. 그 값에 해당하는 자료가 실제로 존재하는지, 요청자가 볼 수 있는지는 후속 업무 처리 단계가 판단한다.

이 책임을 섞으면 경로표가 업무 규칙으로 가득 찬다. 라우터가 자료 존재 여부까지 모두 확인하면 목적지 선택과 실제 작업의 경계가 흐려진다. 반대로 문자열 모양만 맞추고 값을 잃어버리면 처리자는 어느 대상을 다뤄야 할지 모른다. 패턴을 고르고 동적 값을 안전하게 넘기는 것까지가 라우팅의 완결된 역할이다.

라우팅 결정 절차

동작과 경로를 결합해 처리자를 선택하는 과정



라우팅은 업무를 실행하지 않고 하나의 처리자를 선택한다

3.6 라우팅 실패 응답

경로표에서 맞는 줄을 찾지 못했다고 요청이 사라져서는 안 된다. 클라이언트는 서버가 아직 처리 중인지, 메시지가 도착하지 않았는지, 목적지가 틀렸는지 알 수 없기 때문이다. 라우터는 업무 처리자를 호출하지 않고도 알맞은 처리 진입점이 없다는 응답을 만들어 한 차례의 통신을 종료해야 한다.

이때 경로 자체가 없는 경우와 경로는 있지만 동작이 맞지 않는 경우를 구분하면 도움이 된다. 첫 경우에는 목적지 표기를 다시 확인해야 하고, 둘째 경우에는 같은 목적지에서 지원하는 행동을 확인해야 한다. 두 문제를 모두 내부 실패라고 답하면 클라이언트는 자신이 고칠 수 있는 요청 문제를 서버 장애로 오해한다.

라우팅 실패에 응답하는 것은 오류 전달뿐 아니라 시스템 안정성과 연결된다. 명확한 실패는 불필요한 재시도와 긴 기다림을 줄인다. 서버 역시 실행하지 않은 업무와 실행하다 실패한 업무를 구별해 관찰할 수 있다. 라우팅 단계의 실패를 그 자리에서 닫으면 요청 처리 주기의 어느 경계까지 통과했는지가 분명하게 남는다.

3.7 중복 라우팅 규칙과 우선순위

경로 변수는 편리하지만 고정 경로와 겹칠 수 있다. 책 식별자가 오는 자리에 새로 만든 고정 단어가 들어가면, 라우터는 그것을 특별한 처리 진입점으로 볼지 일반 식별자로 볼지 결정해야 한다. 두 규칙이 모두 맞는 상태에서 단순히 먼저 등록된 줄을 고르면 파일 순서가 바뀌었을 때 다른 처리자가 선택될 수 있다.

예측 가능한 방법은 구체적인 규칙을 우선하는 것이다. 고정된 단어가 더 많은 경로를 먼저 판단하고, 넓게 잡는 동적 규칙은 그다음에 둔다. 더 좋은 방법은 가능하면 처음부터 의미가 겹치지 않도록 경로 모양을 설계하는 것이다. 우선순위 규칙은 필요하지만, 사람이 표를 읽고도 한 요청이 어디로 갈지 분명해야 한다.

세 번째 장을 마치면 라우팅은 동작과 경로를 경로표에 대조하고, 동적 값을 꺼내, 하나의 처리자를 선택하거나 명시적인 실패 응답으로 닫는 과정이 된다. 라우터는 업무를 직접 처리하지 않는다. 이 단순한 책임이 지켜져야 다음 장에서 처리자가 받은 요청을 실제 결과로 바꾸는 모습을 별도로 관찰할 수 있다.

4.1 요청 처리자의 책임

라우터가 알맞은 경로를 골랐다고 요청 처리가 끝난 것은 아니다. 선택된 처리 진입점에는 실제 일을 맡는 처리자가 있다. 처리자는 요청에서 필요한 값을 받아 자료를 조회하거나 계산하거나 상태 변경을 시도한다. 그리고 작업 결과를 외부 계약에 맞는 응답 데이터로 정리한다. 흔히 핸들러라고도 부르는 이 부분이 요청의 업무 의미를 실현한다.

처리자는 경로표 전체를 다시 살필 필요가 없다. 자신에게 요청이 왔다는 것은 라우터가 동작과 경로를 이미 맞췄다는 뜻이다. 처리자는 추출된 경로 값과 헤더의 맥락, 본문의 업무 데이터를 입력으로 받는다. 각 단계가 앞선 책임을 되풀이하지 않으면 흐름이 짧아지고, 문제가 생겼을 때 어느 경계의 판단이 잘못되었는지 찾기 쉬워진다.

작은 서버에서는 처리자가 모든 작업을 직접 할 수 있다. 다만 이 책은 처리자 안의 함수 분해나 모듈 구조를 깊게 다루지 않는다. 여기서 중요한 것은 요청 경계에서 받은 값을 실제 결과로 바꾸고, 그 결과를 응답 형태로 내보내는 한 구간이다. 내부 조직보다 입출력 경계를 중심으로 보면 특정 기술 없이도 처리 흐름을 설명할 수 있다.

4.2 요청 맥락의 구성과 전달

처리자가 필요한 값은 요청의 여러 칸에 흩어져 있다. 대상 식별자는 경로에서, 통신 맥락은 헤더에서, 업무 데이터는 본문에서 왔을 수 있다. 서버는 이 값을 한 요청의 맥락으로 묶어 처리자에게 넘긴다. 처리자는 값이 어디에서 파싱되었는지 매번 다시 찾지 않고, 이미 구분된 정보를 사용해 자신의 일을 수행한다.

요청 맥락은 특히 여러 요청이 함께 들어올 때 중요하다. 첫 요청의 대상 식별자와 둘째 요청의 본문이 섞이면 각 단계가 개별적으로 정상이어도 잘못된 결과가 나온다. 따라서 맥락은 한 요청의 처리 주기 동안 분리되어 있어야 한다. 처리 중 다른 작업을 기다리더라도 다시 시작했을 때 같은 요청의 값을 이어서 사용해야 한다.

맥락에 모든 서버 정보를 넣을 필요는 없다. 한 요청을 이해하고 처리하는 데 필요한 값만 담아야 한다. 서버 전체에서 공유하는 설정이나 여러 요청에 공통인 자료와, 이번 요청에만 속한 값을 구분하면 수명이 분명해진다. 요청이 끝날 때 함께 버려도 되는 정보가 무엇인지 알 수 있고, 다음 요청으로 우연히 남는 상태를 줄일 수 있다.

4.3 내부 처리 결과와 응답의 구분

처리자가 자료를 찾았다고 해서 곧바로 응답이 완성되는 것은 아니다. 내부에서 사용하는 결과는 서버가 작업하기 편한 모양일 수 있고, 외부에 약속한 응답은 클라이언트가 이해하기 쉬운 다른 모양일 수 있다. 처리자는 업무 결과를 받은 뒤 어떤 의미의 답인지 정하고, 공개할 값만 골라 응답 재료로 바꾼다.

이 변환은 불필요한 복사가 아니다. 내부 자료에는 외부에 보여 주면 안 되는 값이 있거나, 구현에만 필요한 세부 구조가 포함될 수 있다. 내부 이름을 그대로 응답에 쓰면 저장 방식의 작은 변경이 외부 계약까지 바꾼다. 외부 응답 모양을 별도로 두면 같은 의미를 유지하면서 내부 구현을 고칠 수 있고, 공개 범위도 분명하게 관리할 수 있다.

실패 결과도 변환이 필요하다. 자료를 찾지 못한 경우, 요청 조건이 맞지 않는 경우, 예상하지 못한 내부 문제는 클라이언트가 취할 행동이 서로 다르다. 처리자는 내부 예외 문장을 그대로 내보내지 않고 외부에서 이해할 수 있는 실패 의미로 바꾼다. 작업 결과와 응답 사이의 이 단계가 서버 내부와 외부 계약을 다시 한번 나누는 경계다.

4.4 응답 메시지의 구성

응답은 서버가 처리 결과를 클라이언트에게 전달하는 구조화된 메시지다. 가장 먼저 작업이 어떤 종류로 끝났는지 나타내는 상태가 있고, 응답을 해석하는 데 필요한 부가 정보가 있으며, 필요하면 실제 결과를 담은 본문이 따른다. 요청과 마찬가지로 각 칸의 책임이 다르기 때문에 클라이언트는 전체 문장을 추측하지 않고 결과를 단계적으로 읽을 수 있다.

성공한 응답에는 조회한 자료나 새로 만들어진 대상의 정보가 들어갈 수 있다. 하지만 성공했다고 언제나 본문이 필요한 것은 아니다. 요청한 변경이 끝났다는 상태만으로 충분한 경우도 있다. 실패 응답 역시 자세한 내부 보고서보다 실패 종류와 안전한 설명을 제공해야 한다. 클라이언트는 상태를 보고 다시 시도할지, 요청을 고칠지, 사용자에게 안내할지 결정한다.

응답을 만드는 순간 한 요청의 처리 주기가 닫힌다. 서버는 이 응답을 원래 요청의 연결로 반환하고, 요청에만 속한 맥락을 정리한다. 클라이언트가 응답을 받으면 처음 보낸 의도와 결과를 연결해 다음 화면이나 작업을 선택한다. 좋은 응답은 단지 데이터 전달 형식에 그치지 않고, 요청이 어디까지 처리되었고 이제 무엇을 할 수 있는지 알려 주는 계약이다.

4.5 상태와 본문의 의미 일치

응답의 상태와 본문이 서로 다른 이야기를 하면 클라이언트는 어느 쪽을 믿어야 할지 알 수 없다. 상태는 성공을 나타내는데 본문에는 실패 문장이 들어 있거나, 실패 상태인데 정상 자료가 함께 오면 프로그램마다 다르게 해석할 수 있다. 화면에는 성공처럼 보이지만 실제 값은 반영되지 않는 혼란도 생긴다.

클라이언트는 보통 상태를 먼저 보고 처리 분기를 고른다. 성공이면 본문을 정상 자료로 읽고, 실패면 오류 형식으로 읽는다. 따라서 상태가 정한 의미와 본문의 모양이 일치해야 한다. 본문 속 특정 글자를 다시 찾아 성공 여부를 판정하게 만들면 외부 계약이 불명확해지고, 설명 문장을 조금 바꾸는 일도 동작 변경이 된다.

일관된 응답은 테스트와 관찰도 단순하게 한다. 어떤 결과에서 어떤 상태와 본문이 나와야 하는지 표로 적을 수 있고, 실제 응답이 그 표와 맞는지 확인할 수 있다. 서버 내부에서는 여러 실패가 일어나더라도 바깥으로 나가는 순간에는 정해진 의미로 정돈되어야 한다. 응답의 각 칸은 독립적으로 존재하지만 같은 처리 결과를 함께 설명한다.

4.6 오류 응답의 정보 노출 범위

예상하지 못한 문제가 생기면 서버가 가진 오류 문장을 그대로 응답에 넣고 싶을 수 있다. 개발 중에는 자세해 보여 유용하지만, 내부 파일 위치와 자료 구조와 작업 순서가 바깥으로 드러날 수 있다. 클라이언트에게는 고칠 수 없는 세부이 많고, 서버 구현이 바뀔 때마다 오류 모양도 달라져 안정된 계약이 되지 못한다.

외부 응답은 후속 처리를 고르는 데 필요한 의미를 제공하면 된다. 요청 형식을 고쳐야 하는지, 해당 대상을 찾을 수 없는지, 잠시 뒤 다시 시도할 수 있는지처럼 클라이언트의 결정과 연결되는 정보가 중요하다. 더 자세한 내부 원인은 서버 안의 관찰 자료에 남길 수 있지만, 그것을 외부 답과 동일하게 취급하지 않는다.

정보를 줄인다고 모든 실패를 같은 말로 뭉개서는 안 된다. 지나치게 모호한 응답 역시 클라이언트가 문제를 해결하지 못하게 한다. 안전한 오류 응답은 숨김과 설명 사이에서 계약에 필요한 수준을 고른다. 내부 구조를 노출하지 않으면서 실패 종류와 요청자가 할 수 있는 행동을 일관되게 알려 주는 것이 목표다.

4.7 요청 처리와 응답 생성 절차

처리 흐름을 다시 잇으면 라우터가 처리자를 고른 뒤 요청 맥락을 넘기고, 처리자가 실제 작업을 수행하며, 내부 결과를 외부 응답 모양으로 바꾸는 순서가 된다. 서버는 완성된 응답을 원래 요청에 반환하고 요청 전용 맥락을 정리한다. 이 흐름은 성공과 실패 모두에서 마지막 응답을 만들어야 끝난다.

각 단계의 책임이 분명하면 문제의 위치도 구분된다. 처리 진입점을 못 찾으면 라우팅 문제이고, 알맞은 처리자에 도착했지만 자료 조건이 맞지 않으면 업무 결과의 문제다. 결과는 맞는데 상태와 본문이 모순된다면 응답 변환의 문제다. 모든 실패를 처리자 한곳에 모으지 않으면 관찰할 질문이 구체적이 된다.

지금까지는 요청 하나가 서버 안을 지나가는 모습을 주로 보았다. 실제 서버 운영 환경에는 여러 요청이 거의 동시에 도착한다. 각 요청의 맥락과 응답 대응 관계를 유지하면서 제한된 처리 능력을 나누어 써야 한다. 다음 장에서는 한 요청의 단계는 그대로 둔 채, 여러 시간선이 겹칠 때 무엇이 달라지는지 살펴본다.

5.1 동시 요청의 정의

사용자 두 명이 거의 같은 시각에 버튼을 누르면 서버에는 둘 이상의 요청이 겹쳐 도착한다. 정확히 같은 찰나에 도착하지 않았더라도 첫 요청이 끝나기 전에 둘째 요청이 들어오면 서버 안에는 두 시간선이 함께 존재한다. 이처럼 여러 요청의 처리 주기가 겹치는 상황을 동시 요청이라고 생각할 수 있다.

겹친 요청은 도착한 순서대로 끝나지 않을 수 있다. 첫 요청이 느린 외부 작업을 기다리는 동안 둘째 요청이 짧은 계산을 마치면 둘째 응답이 먼저 돌아간다. 이것은 순서가 뒤집힌 오류가 아니다. 각 응답이 자신의 요청으로 정확히 돌아가고, 서로 공유하는 상태를 잘못 덮어쓰지 않는다면 완료 순서가 다른 것은 자연스럽다.

문제는 한 요청만 생각하고 만든 코드가 여러 시간선에서도 같은 의미를 유지하는지다. 요청마다 달라야 할 값을 공용 공간에 두면 다른 요청이 중간에 바꿀 수 있다. 처리 능력보다 많은 요청이 오면 누군가는 기다려야 한다. 동시성을 이해하려면 빠르게 만드는 기술보다 먼저, 겹친 시간선에서 무엇을 분리하고 무엇을 조정해야 하는지 살펴야 한다.

5.2 작업자와 요청 작업의 관계

서버에서 요청 작업을 실행하는 제한된 자원을 작업자라고 한다. 작업자 하나는 한 시점에 하나의 계산 작업을 수행하며, 여러 작업자가 있으면 여러 요청을 병행할 수 있다. 작업자 수는 계산 자원과 메모리 한계의 영향을 받는다. 실행 대기 요청 수가 작업자 수를 초과하면 새 요청 작업은 즉시 실행되지 않고 대기열에서 실행 가능 시점을 기다린다.

작업자 수를 무제한으로 늘리면 작업 전환 비용과 메모리 사용량이 증가한다. 반대로 작업자 수가 지나치게 적으면 사용할 수 있는 계산 자원이 남아 있어도 대기 시간이 길어질 수 있다. 적절한 작업자 수는 요청 작업이 계산 중심인지 외부 결과 대기 중심인지, 서버가 사용할 수 있는 자원이 얼마인지에 따라 달라진다. 설정값 자체보다 작업 특성과 자원 사용을 함께 측정하는 것이 중요하다.

요청은 수행해야 할 작업이고 작업자는 그 작업을 실행하는 자원이므로 두 개념을 구분해야 한다. 외부 시스템의 응답을 기다리는 동안 작업자가 계속 점유되는지, 실행 가능한 다른 요청을 처리할 수 있는지에 따라 동시 처리 능력이 달라진다. 아직 작업자를 배정받지 못한 요청은 대기열에서 관리한다. 다음 절에서는 대기열의 역할, 대기 시간, 수용 한계를 설명한다.

5.3 처리 대기열과 대기 시간

모든 작업자가 작업 중일 때 도착한 요청을 곧바로 버리지 않으려면 잠시 기다릴 곳이 필요하다. 대기열은 실행을 기다리는 작업을 순서 있게 보관한다. 작업자가 하나 비면 대기열에서 다음 작업을 꺼내 처리한다. 짧은 순간에 요청이 몰려도 곧 줄어든다면 대기열이 일시적인 유입 증가를 수용해 서버는 안정적으로 응답할 수 있다.

하지만 대기열이 있다고 처리 능력이 늘어나는 것은 아니다. 들어오는 속도가 끝나는 속도보다 계속 빠르면 줄은 길어지고, 요청은 실행되기도 전에 오래 기다린다. 대기열을 무한히 키우면 메모리를 소모하고 클라이언트의 최대 기다림 시간을 넘긴 작업까지 뒤늦게 수행할 수 있다. 기다림을 저장하는 공간에는 반드시 수용 한계가 필요하다.

대기열 길이와 머문 시간은 서버가 밀리고 있다는 신호다. 실행 시간만 보면 처리자 자체는 빨라 보여도, 대기 시간이 더해지면 사용자에게 돌아가는 응답은 느리다. 요청의 전체 처리 주기를 보려면 줄에서 기다린 시간과 실제 일한 시간을 나누어야 한다. 대기열은 부하를 숨기는 장치가 아니라 처리 능력과 유입량의 차이를 눈에 보이게 하는 경계다.

5.4 요청 맥락 격리와 공유 상태

동시 요청 처리의 첫 원칙은 각 요청의 맥락을 분리하는 것이다. 대상 식별자, 본문, 요청자를 구분하는 정보, 중간 계산 결과가 한 요청의 처리 주기 안에 머물러야 한다. 둘 이상의 작업이 같은 변수를 임시 저장소처럼 쓰면 나중 요청이 먼저 요청의 값을 덮어쓸 수 있고, 잘못된 대상에 응답하는 심각한 문제가 생긴다.

요청마다 분리할 수 없는 값도 있다. 같은 재고 수량이나 하나의 좌석처럼 여러 요청이 함께 바꾸려는 상태가 그 예다. 이때는 읽기와 변경의 순서를 조정해 두 요청이 모두 이전 값을 보고 성공했다고 판단하지 않게 해야 한다. 구체적인 저장 기술보다 중요한 원리는 공유 상태를 바꾸는 한 동작이 중간에서 다른 작업과 모순되게 섞이지 않도록 보호하는 것이다.

동시 처리 성능을 높인다는 이유로 격리를 포기할 수는 없다. 잘못된 응답을 빠르게 반환하는 서버는 좋은 서버가 아니다. 먼저 각 요청이 자신의 입력과 결과를 보존하고, 공유 상태 변경은 예측 가능한 규칙을 따르게 해야 한다. 그 위에서 작업자 수와 대기열을 조정하면 처리량과 정확성을 함께 다룰 수 있다.

동시 요청 처리 시간선

작업자 두 개와 대기열을 사용하는 처리 예시

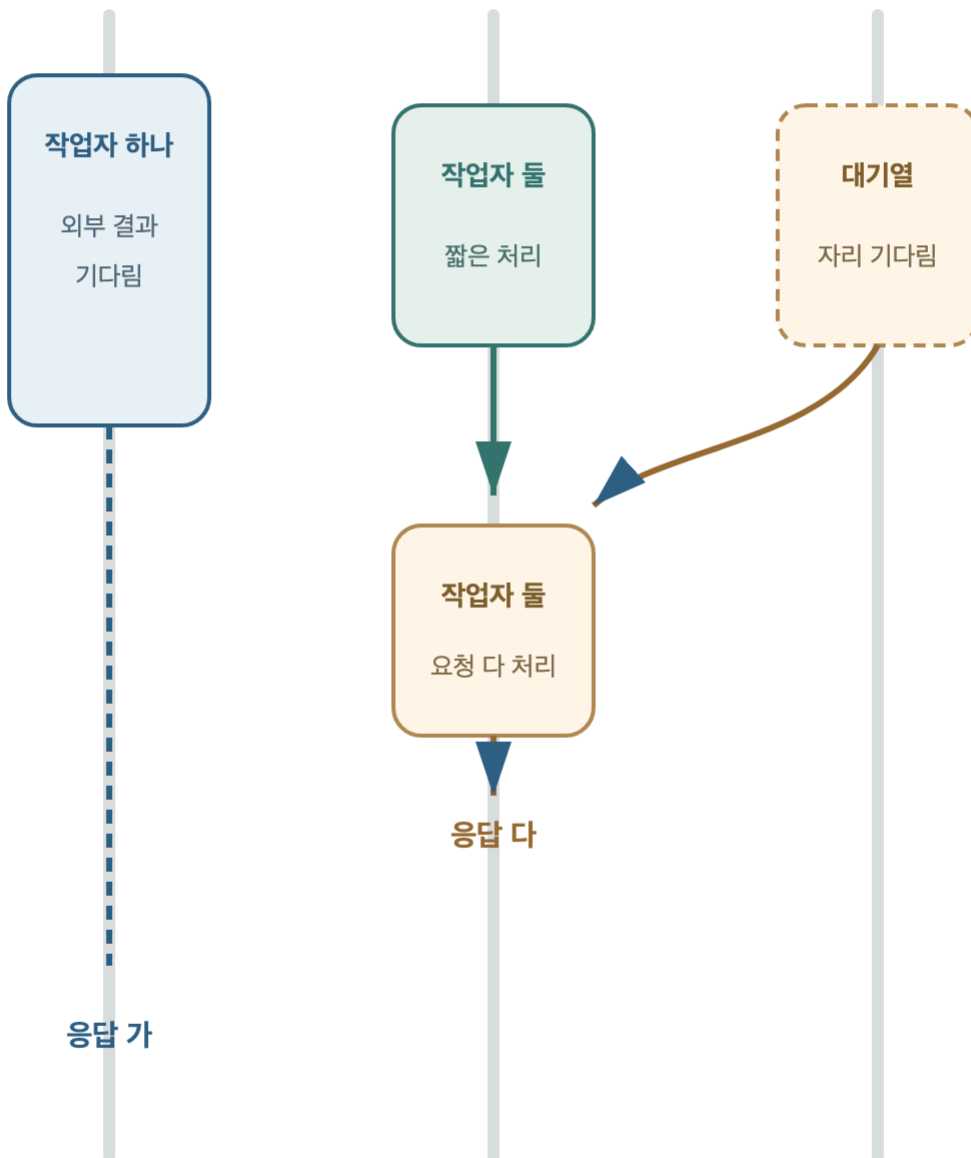
도착

시간 ↓

요청 가

요청 나

요청 다



응답 가

응답 나

완료 순서와 관계없이 각 응답은 원래 요청과 연결된다

5.6 대기 작업과 계산 작업의 차이

요청 처리 시간에는 서버가 직접 계산하는 시간과 다른 곳의 결과를 기다리는 시간이 함께 들어 있다. 계산 중에는 실행 자원을 적극적으로 사용하지만, 외부 자료가 돌아오기를 기다리는 동안에는 실제 계산을 거의 하지 않을 수 있다. 두 시간을 같은 느낌으로 보면 서버가 왜 많은 요청을 받지 못하는지 정확히 설명하기 어렵다.

작업자가 기다리는 동안 다른 일을 전혀 맡지 못하면 이를 차단된 상태로 볼 수 있다. 작업자 수가 적고 외부 대기가 길면 대부분의 작업자가 멈춘 채 새 요청은 대기열에 쌓인다. 반면 기다림을 표시해 두고 같은 실행 자원이 다른 준비된 작업을 진행할 수 있는 방식도 있다. 어느 방식이든 요청 맥락이 다시 시작될 때 정확히 이어져야 한다.

그렇다고 모든 작업을 복잡한 비차단 방식으로 바꾸는 것이 정답은 아니다. 짧은 계산 중심 서버라면 단순한 실행 방식이 이해하고 운영하기 쉬울 수 있다. 먼저 실제 시간이 계산에 쓰이는지 기다림에 쓰이는지 관찰하고, 병목에 맞는 방법을 고른다. 동시 처리 설계는 유행하는 구조를 따르는 일이 아니라 제한된 자원을 작업 특성에 맞추는 판단이다.

5.7 과부하와 수용 한계

요청이 잠깐 물리는 정도는 대기열이 흡수할 수 있다. 그러나 들어오는 속도가 오랫동안 처리 속도를 넘으면 언젠가는 한계에 도달한다. 끝없이 받아 두면 모든 요청의 대기 시간이 늘고, 클라이언트가 이미 기다림을 포기한 작업까지 서버는 뒤늦게 수행한다. 결국 자원은 소모되지만 유효한 응답은 줄어드는 상태가 된다.

이때는 감당할 수 있는 대기 수와 시간을 정하고, 넘는 요청에는 지금 처리할 수 없다는 명확한 응답을 빠르게 반환하는 편이 낫다. 일부 요청은 실패하지만 서버 전체가 멈추는 것을 막고, 클라이언트도 무작정 기다리지 않고 나중에 시도할지 결정할 수 있다. 거절은 무책임한 포기가 아니라 시스템의 실제 능력을 계약에 반영하는 경계다.

다섯 번째 장의 핵심은 더 많은 요청을 무조건 동시에 실행하는 것이 아니다. 요청별 맥락을 분리하고, 공유 상태 변경을 조정하며, 제한된 작업자와 대기열을 관찰하는 것이다. 계산과 기다림의 차이를 알고 수용 한계를 두면 소규모 서버도 요청 증가 상황에서 예측 가능한 응답을 유지할 수 있다. 다음 장에서는 요청 사이에 무엇을 남기지 않아야 여러 서버가 같은 역할을 나눌 수 있는지 살핀다.

6.1 무상태 서버의 정의

무상태 서버는 업무 데이터를 전혀 저장하지 않는 서버를 뜻하지 않는다. 핵심은 특정 서버 인스턴스의 로컬 상태가 다음 요청의 필수 처리 조건이 되지 않도록 설계하는 데 있다. 계정, 주문, 대여처럼 요청 사이에 유지해야 하는 업무 상태는 존재할 수 있다. 다만 이러한 상태를 특정 실행 인스턴스의 수명에 종속시키지 않아야 한다.

각 요청은 처리에 필요한 정보를 직접 제공하거나 모든 서버가 접근할 수 있는 외부 상태를 식별해야 한다. 어느 서버 인스턴스가 요청을 받더라도 동일한 계약과 외부 상태를 기준으로 같은 의미의 결과를 만들 수 있어야 한다. 이 조건이 충족되면 후속 요청을 이전 요청과 같은 서버로 강제할 필요가 없다. 서버 교체와 요청 분산도 단순해진다.

무상태 원칙은 캐시나 연결 정보 같은 모든 서버 내부 데이터를 금지하지 않는다. 로컬 데이터가 사라지더라도 업무 상태와 다음 요청의 필수 조건이 유지되는지가 판단 기준이다. 요청 간에 지속해야 하는 업무 상태와 요청 종료 후 제거할 수 있는 임시 맥락을 수명에 따라 분리해야 한다. 이 구분이 무상태 서버 설계의 출발점이다.

6.2 서버 인스턴스 종속 상태의 문제

첫 요청의 사용자 확인 결과를 한 서버 인스턴스의 로컬 상태에만 저장한 경우를 가정한다. 다음 요청이 같은 서버로 오면 문제없이 이어지지만, 다른 서버가 받으면 이전 정보를 찾을 수 없다. 그래서 모든 후속 요청을 처음 서버로만 보내는 별도 규칙이 필요해진다. 서버를 추가해도 요청을 자유롭게 나누지 못한다.

재시작도 문제가 된다. 서버 프로그램은 수정이나 장애 복구 때문에 다시 실행될 수 있다. 업무에 필요한 상태가 실행 공간에만 있었다면 재시작과 함께 사라지고, 사용자는 갑자기 이전 행동을 이어 갈 수 없게 된다. 로컬 상태를 복제하려고 해도 여러 서버의 값이 언제 같아지는지 관리하는 새로운 문제가 생긴다.

한 서버 인스턴스의 로컬 상태이 빠르고 간단하다는 장점은 있다. 그러나 그 값을 잃어도 되는지, 다른 서버가 몰라도 되는지를 먼저 물어야 한다. 요청 사이의 업무 연속성에 꼭 필요한 값이라면 특정 실행 공간보다 안정된 외부 위치에 두는 편이 안전하다. 무상태 원칙은 서버를 잊게 만드는 것이 아니라 지속 상태의 저장 위치를 명확히 정하는 방법이다.

6.3 업무 상태의 외부 저장

무상태 서버라고 해서 사용자의 행동 기록이나 업무 자료가 사라지는 것은 아니다. 오래 남아야 하는 상태는 여러 서버가 접근할 수 있는 외부 저장소로 옮긴다. 서버는 요청을 받을 때 필요한 상태를 읽고, 작업 결과에 따라 변경한 뒤, 응답을 만들고 요청 전용 맥락을 버린다. 다음 요청은 어느 서버가 받더라도 같은 외부 상태에서 출발할 수 있다.

이 분리는 수명의 차이를 반영한다. 서버 인스턴스는 배포나 장애 때문에 언제든지 교체될 수 있지만, 사용자가 만든 자료는 서버 인스턴스보다 긴 기간 유지되어야 한다. 실행 공간과 업무 상태를 묶지 않으면 서버를 재시작해도 업무 상태는 유지된다. 새 서버를 추가해도 별도의 로컬 상태를 복사할 필요 없이 같은 저장소를 바라보게 할 수 있다.

물론 외부 저장소 자체의 일관성과 장애를 다루는 설계가 필요하다. 무상태 원칙이 모든 문제를 없애지는 않는다. 대신 상태의 위치를 명시해 어디가 진실의 책임을 갖는지 분명하게 만든다. 서버는 요청을 처리하는 교체 가능한 작업자가 되고, 장기 상태는 그 수명에 맞는 별도 경계에서 관리된다.

6.4 요청 식별자와 업무 상태의 구분

동시에 여러 요청이 오면 어느 처리 기록과 응답이 같은 요청에 속하는지 구분할 표식이 유용하다. 요청 식별자는 한 번의 요청-응답 처리에 같은 값을 붙여 시간선을 따라가게 한다. 서버 안의 여러 단계를 지나더라도 같은 표식을 유지하면 요청과 응답의 대응 관계를 관찰할 수 있다.

하지만 이 표식은 사용자의 업무 상태와 다르다. 요청이 끝나면 더는 다음 요청의 필수 입력으로 쓰지 않아도 된다. 반면 대여 여부나 잔액처럼 다음 행동에 영향을 주는 값은 오래 유지되어야 한다. 둘을 같은 저장 방식으로 다루면 일시적 정보가 불필요하게 쌓이거나, 장기 상태를 너무 빨리 버리는 문제가 생긴다.

정보의 수명을 질문하면 자리가 보인다. 이번 요청을 처리하는 동안만 필요한가, 다음 요청에서도 의미가 있는가, 서버가 교체되어도 남아야 하는가를 차례로 묻는다. 요청 표식은 첫 질문에 가깝고 업무 상태는 마지막 질문까지 통과한다. 무상태 서버는 모든 맥락을 없애는 대신 서로 다른 수명을 정확히 분리한다.

6.5 무상태 서버의 수평 확장

요청이 늘어 한 서버의 처리 능력이 부족해지면 같은 역할의 서버를 하나 더 둘 수 있다. 앞단의 분배자는 들어온 요청을 둘 중 여유 있는 곳으로 보낸다. 각 요청이 특정 서버의 이전 로컬 상태에 의존하지 않는다면 어느 쪽으로 가도 같은 외부 상태와 계약을 바탕으로 처리된다. 이를 서버를 옆으로 늘리는 수평 확장이라고 한다.

수평 확장의 장점은 처리량만이 아니다. 서버 하나가 멈춰도 다른 서버가 새 요청을 받을 수 있고, 수정한 서버를 차례로 교체할 수도 있다. 특정 사용자가 특정 인스턴스에 묶여 있지 않으므로 요청 분배 규칙이 단순해진다. 교체 가능한 실행 인스턴스와 지속되는 업무 상태의 경계가 운영 유연성을 만든다.

다만 서버 수만 늘리면 모든 병목이 해결되는 것은 아니다. 모두가 같은 외부 저장소에 몰리면 그곳이 새로운 한계가 될 수 있고, 요청을 나누는 앞단도 충분한 능력이 필요하다. 무상태 원칙은 확장의 필요조건에 가깝지 자동 해답은 아니다. 그래도 특정 서버 로컬 상태를 복제하는 문제를 없애 가장 기본적인 분산을 가능하게 한다.

6.6 재시도와 중복 요청 처리

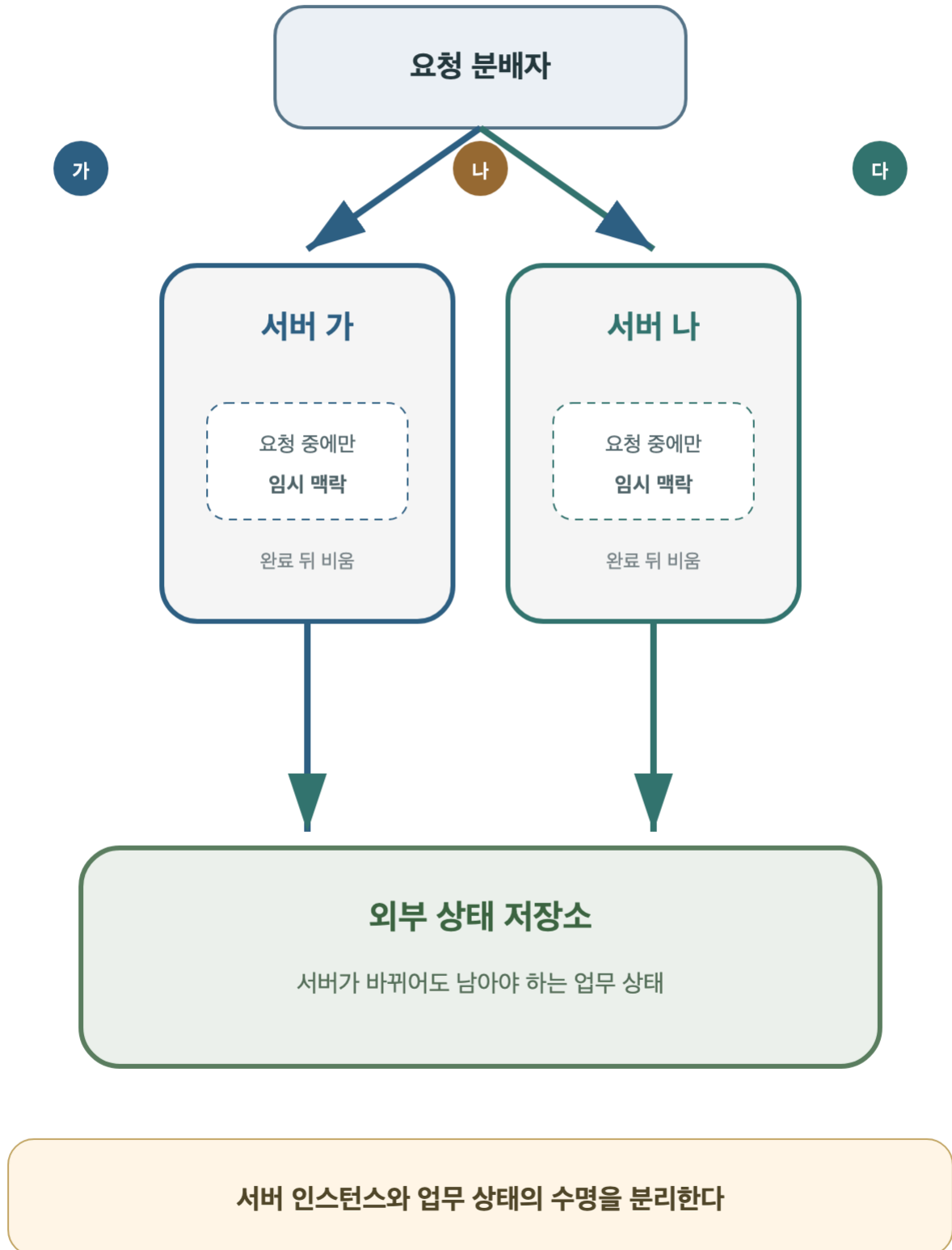
클라이언트가 응답을 받지 못했다고 서버 작업까지 실패한 것은 아니다. 서버가 변경을 마친 뒤 응답이 전송 과정에서 유실되었을 수 있다. 클라이언트는 결과를 모르므로 같은 요청을 다시 보낼 수 있다. 조회라면 같은 결과를 한 번 더 읽는 정도지만, 결제나 차감처럼 상태를 바꾸는 작업은 두 번 실행되면 의미가 달라진다.

무상태 원칙은 이전 요청을 무조건 잊고 같은 변경을 다시 수행하라는 뜻이 아니다. 장기적인 업무 결과는 외부 상태에 남아 있으므로, 같은 시도를 구분할 표식이나 이미 처리된 결과를 확인하는 규칙을 둘 수 있다. 서버 인스턴스의 임시 상태이 아니라 공유되는 업무 경계에서 중복을 판단해야 어느 서버가 재시도를 받아도 같은 결론을 낼 수 있다.

모든 요청에 복잡한 중복 방지 장치를 넣을 필요는 없다. 먼저 같은 요청을 여러 번 실행해도 결과가 같은지 살핀다. 같다면 단순하게 다시 처리할 수 있고, 다르다면 중복이 실제 손실을 만드는 범위에 필요한 규칙을 둔다. 재시도 문제는 무상태를 포기할 이유가 아니라, 일시적 서버 상태와 지속되는 업무 결과를 더 정확히 구분하게 하는 사례다.

무상태 서버 구성

여러 서버가 외부 상태 저장소를 공유하는 구조



7.1 전체 요청 처리 단계

한 요청의 전체 처리 단계는 수신, 해석, 라우팅, 업무 처리, 응답 생성, 반환 순서로 구성된다. 클라이언트가 구조화된 요청을 전송하면 서버 경계가 메시지 형식을 해석한다. 라우터는 동작과 경로를 경로표와 비교하고 동적 경로 값을 추출해 처리자를 선택한다. 처리자는 요청 맥락을 사용해 업무를 수행하고 내부 결과를 외부 계약에 맞는 응답으로 변환한다. 응답이 클라이언트에 반환되면 해당 요청 처리가 종료된다.

각 단계에는 독립적인 실패 조건이 있다. 메시지 구조를 해석할 수 없거나 라우팅 규칙이 없으면 업무 처리자를 호출하기 전에 실패 응답을 생성한다. 처리 중 업무 조건이 충족되지 않거나 예상하지 못한 문제가 발생하면 내부 결과를 안전한 오류 응답으로 변환한다. 성공 경로뿐 아니라 모든 실패 경로도 명시적인 응답 또는 시간 초과 판단으로 종료되어야 한다.

여러 요청이 동시에 도착하면 동일한 처리 단계가 요청별 시간선에서 병행된다. 각 요청 맥락은 분리하고, 제한된 작업자와 대기열을 공유하며, 공용 상태 변경은 일관성을 유지하도록 조정한다. 요청 간에 지속해야 하는 업무 상태는 특정 서버 인스턴스가 아니라 외부 상태 경계에 저장한다. 동시성과 무상태 원칙은 기본 요청 처리 흐름에 격리, 자원 한계, 상태 수명 조건을 추가한다.

7.2 조회 요청의 정상 처리 사례

클라이언트가 특정 책의 정보를 조회하는 요청을 전송하는 상황을 가정한다. 요청의 동작은 조회 의도를 나타내고, 경로에는 책을 가리키는 고정 구간과 식별자가 들어 있다. 서버 경계는 메시지를 읽고 라우터는 경로표에서 같은 동작과 모양을 찾는다. 동적 구간의 식별자를 꺼내 요청 맥락에 담아 조회 처리자로 넘긴다.

처리자는 식별자를 사용해 외부 상태 저장소에서 책 자료를 찾는다. 자료가 존재하면 내부 결과에서 외부에 공개하기로 한 제목과 설명 같은 값만 골라 응답 본문으로 바꾼다. 성공 상태와 본문 형식 설명을 함께 넣고, 완성된 응답을 원래 요청에 반환한다. 요청에만 쓰인 맥락은 처리가 끝난 뒤 남겨 둘 필요가 없다.

이 사례에서 라우터는 책의 존재를 판단하지 않았고, 저장소는 응답 형식을 알지 못했다. 라우터는 처리 진입점을 골랐고, 처리자는 업무 결과를 얻었으며, 응답 변환은 외부 계약을 만들었다. 한 번의 조회가 짧게 끝나더라도 책임을 나누어 보면 각 단계가 무엇을 보장하는지 분명하다. 같은 경로에서 실패가 생겨도 어느 단계까지 성공했는지 설명할 수 있다.

7.3 존재하지 않는 경로의 처리 사례

클라이언트가 서버에서 공개하지 않은 경로로 조회 요청을 전송하는 상황을 가정한다. 메시지의 기본 구조는 읽을 수 있지만 라우터가 경로표를 비교해도 같은 모양을 찾지 못한다. 이 요청은 업무 처리자에게 도착하지 않는다. 저장소를 조회하거나 비슷한 경로를 추측해 대신 실행해서도 안 된다.

라우터는 목적지를 찾지 못했다는 안정된 실패 응답을 만든다. 클라이언트는 응답 상태를 보고 경로 표기를 고쳐야 한다는 사실을 알 수 있다. 서버 내부에서는 처리자 실행 시간이 없고 저장소 접근도 없었다. 전체 요청-응답 처리는 성공 사례보다 짧지만, 요청을 받고 의미 있는 응답으로 답았다는 점에서 완전한 처리 흐름이다.

이 짧은 경로는 실패를 가능한 경계 가까이에서 끝내는 장점을 보여 준다. 알 수 없는 요청을 후속 업무 처리 단계로 보내지 않으므로 불필요한 자원을 쓰지 않고, 내부의 우연한 동작을 막는다. 문제의 위치도 라우팅으로 좁혀진다. 실패 흐름을 별도로 그려 두면 성공 코드 사이에 숨은 조건문보다 서버의 실제 계약을 더 정확히 이해할 수 있다.

7.4 동시 요청 처리 사례

조회 요청 두 건과 변경 요청 한 건이 거의 동시에 도착하는 상황을 가정한다. 작업자는 두 명이라 앞선 두 요청이 먼저 실행되고 셋째 요청은 대기열에 머문다. 첫 조회가 외부 결과를 기다리는 사이 둘째 변경이 작업을 마쳐 응답한다. 빈 작업자가 생기면 셋째 조회가 실행되고, 마지막에 첫 조회의 결과가 돌아온다.

완료 순서는 도착 순서와 다르지만 각각의 요청 맥락이 분리되어 있으면 문제없다. 첫 조회의 식별자가 셋째 조회에 섞이지 않고, 둘째 변경 응답은 원래 변경 요청으로 돌아간다. 만약 변경이 두 개였다면 같은 상태를 동시에 바꾸지 않도록 저장 경계에서 순서를 조정해야 한다. 격리 대상과 공유 대상을 구분하는 것이 핵심이다.

대기열이 계속 길어지는지도 함께 본다. 이번 유입 증가가 일시적이고 대기열이 곧 감소하면 정상적인 완충 범위다. 유입이 계속되어 줄이 늘어난다면 수용 한계를 넘기기 전에 일부 요청에 처리 불가 응답을 돌려야 한다. 동시 처리 사례는 요청 하나의 단계에 작업자와 대기열, 격리와 공유 상태라는 두 번째 축을 덧붙여 읽는 연습이다.

7.5 서버 인스턴스 변경 사례

사용자가 첫 요청으로 책을 대여하고 다음 요청으로 대여한 페이지를 조회하는 상황을 가정한다. 첫 요청은 서버 가가 받아 대여 결과를 외부 상태 저장소에 기록하고 응답한다. 그 뒤 분배자는 다음 요청을 서버 나로 보낼 수 있다. 서버 나 는 서버 가의 실행 공간을 볼 수 없지만, 요청의 사용자 표식과 외부 저장소의 대여 상태를 사용해 페이지 접근을 판단한다.

이 흐름에서 첫 요청의 임시 계산 값은 서버 가와 함께 사라져도 된다. 다음 요청에 필요한 업무 사실은 외부 상태에 남았기 때문이다. 서버 가가 재시작되거나 교체되어도 대여 기록은 유지된다. 서버 나 는 같은 계약과 상태 경계를 사용하므로 이전 요청을 직접 처리하지 않았어도 올바른 결과를 만들 수 있다.

외부 상태 저장소는 단순히 데이터를 놓는 장소가 아니라 서버 인스턴스와 다른 수명의 책임자다. 요청 전용 맥락은 짧게 살고, 업무 상태는 서비스가 요구하는 기간 동안 지속된다. 이 수명 분리가 지켜지면 서버를 늘리고 줄이는 운영 변화가 사용자 행동의 연속성을 깨뜨리지 않는다. 무상태 원칙이 실제 요청 처리에서 주는 효과가 여기 드러난다.

7.6 요청 처리 흐름 점검 항목

서버 흐름을 읽을 때 첫째 누가 요청을 시작하는지, 둘째 동작과 경로와 부가 정보와 본문이 어떻게 나뉘는지 묻는다. 셋째 라우터가 어떤 경로표로 처리자를 고르는지, 넷째 처리자가 어떤 입력으로 실제 작업을 수행하는지 확인한다. 이 질문은 외부 경계에서 내부 결과까지 한 요청의 앞부분을 따라간다.

다섯째 내부 결과가 어떤 상태와 본문을 가진 응답으로 바뀌는지, 여섯째 여러 요청의 맥락이 서로 섞이지 않는지 살핀다. 일곱째 작업자가 모두 바쁠 때 요청이 어디서 얼마나 기다리고 수용 한계는 무엇인지 묻는다. 성공 사례뿐 아니라 경로 없음과 처리 실패와 과부하가 각각 어느 응답으로 종료되는지도 함께 그린다.

마지막으로 다음 요청에도 필요한 상태가 어디에 남는지 확인한다. 특정 서버의 실행 공간의 로컬 상태에만 있다면 교체와 확장에서 문제가 생길 수 있다. 외부 상태와 요청 전용 맥락을 나누면 어느 서버가 받아도 같은 계약으로 처리할 수 있다. 여덟 질문은 특정 기술의 설정표가 아니라, 낯선 서버를 만나도 요청의 처리 주기를 놓치지 않게 하는 공통 분석 기준이다.

7.7 핵심 원칙 정리

서버의 요청 처리는 수신 대기에서 시작해 응답 반환으로 종료된다. 클라이언트는 구조화된 요청을 전송하고, 서버는 메시지 형식을 해석하며, 라우터는 처리자를 선택한다. 처리자는 업무 결과를 생성하고 외부 계약에 맞는 응답으로 변환한다. 성공과 실패 모두 명시적인 응답 또는 시간 초과 판단으로 종료되어야 요청-응답의 대응 관계를 유지할 수 있다.

요청이 증가해 처리 시간선이 겹쳐도 기본 계약은 유지되어야 한다. 요청별 맥락과 응답을 분리하고, 공유 상태 변경의 일관성을 보호하며, 작업자와 대기열의 수용 한계를 명시한다. 다음 요청에 필요한 업무 상태는 특정 서버 인스턴스의 로컬 상태가 아니라 외부 상태 경계에 저장한다. 이 구조를 사용하면 서버를 교체하거나 추가해도 동일한 요청 처리 역할을 유지할 수 있다.

서버 구조를 이해하려면 모든 내부 코드를 동시에 분석할 필요는 없다. 요청이 어느 경계를 통과하고, 각 단계가 어떤 입력을 받아 어떤 출력을 생성하며, 실패가 어느 응답으로 변환되는지 순서대로 확인하면 된다. 새로운 기술이나 프레임워크도 수신, 라우팅, 처리, 응답, 동시성, 상태 관리 중 어느 책임을 구현하는지 배치해 볼 수 있다. 서버는 요청 처리가 끝나면 다시 수신 대기 상태로 전환한다.